Network Intrusion Detection System

Machine Learning & Deep Learning on the NSL-KDD dataset

Best model: XGBoost | Accuracy 0.998 | F1 0.997 | ROC AUC 1.000

Goal: automatically classify each network connection as **NORMAL** or **ANOMALY (attack)**, so security teams do not have to inspect millions of connections by hand.

0. What This Project Is

An Intrusion Detection System (IDS) looks at one network connection at a time — who connected, how many bytes were exchanged, which service was used, the error rates, and so on — and decides whether the traffic is normal or an attack. Doing this by hand across millions of daily connections is impossible, so a machine-learning model is trained to do it instantly and consistently.

This report covers the data, the problems encountered, the method, the four models trained, how well they performed (and what each score means), why the model makes its decisions, and what every output file contains.

1. The Data

Two files were provided. Each row describes a single connection using 41 features (e.g. *protocol_type*, *service*, *src_bytes*, *dst_bytes*, *count*, error rates).

File	Rows	Has label?	Used for
Train_data.csv	25,192	Yes (class)	Training + evaluation
Test_data.csv	22,544	No	Final predictions only

The label column *class* has just two values: **normal** and **anomaly**. After cleaning, the data is roughly balanced at about 53% normal / 47% attack.

2. Problems Encountered & How They Were Solved

Problem 1 — No attack sub-types in the data

The original plan wanted to separate attacks into DoS / Probe / U2R / R2L. These CSV files only contain **normal vs anomaly** — the sub-type labels are not present. **Solution:** a binary classifier (normal vs attack) was built. The code can extend to multi-class by changing one line if labelled sub-type data is obtained.

Problem 2 — The test file has no answers

Test_data.csv has no *class* column, so accuracy cannot be measured on it (there is nothing to check against). **Solution:** the labelled Train_data.csv was split 80% / 20%. The model is trained on 80% and scored on the untouched 20% (an honest test). Test_data.csv only receives predictions, saved to a file.

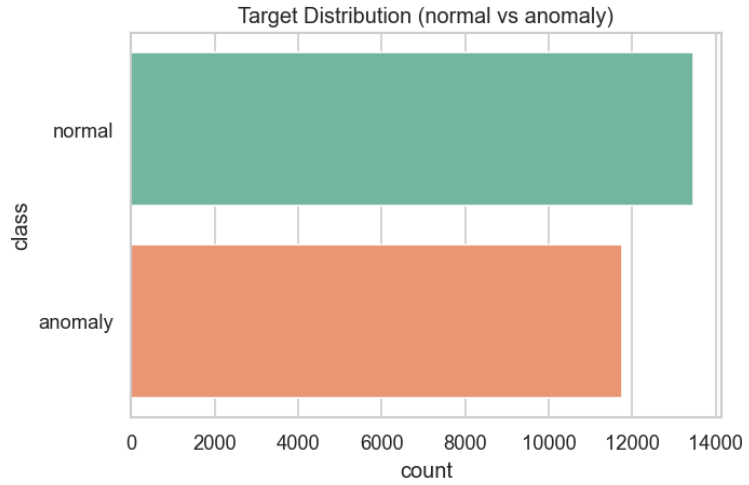
Problem 3 — TensorFlow could not be installed

The planned Keras deep neural network needs TensorFlow, which does not install cleanly on this Python 3.13 environment. **Solution:** scikit-learn's MLPClassifier (a neural network with layers 256 → 128 → 64) was used instead — same concept, different library — and it scored about 99.3% F1.

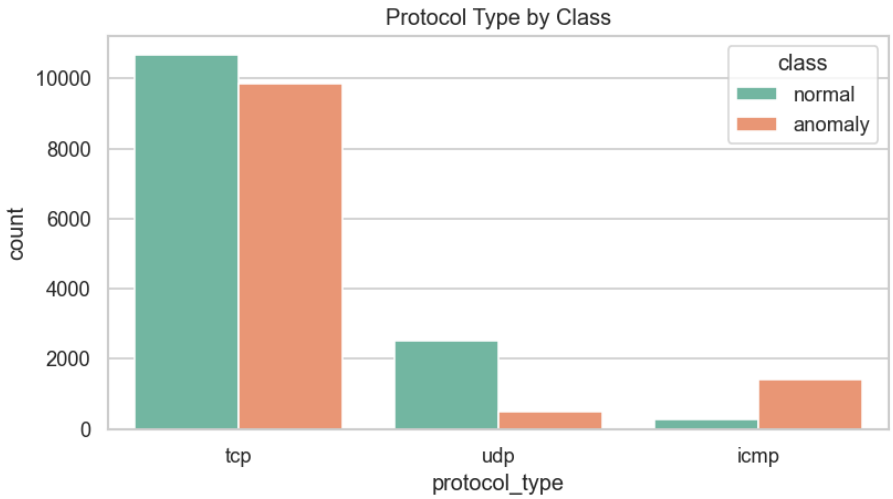
3. What Was Done (Step by Step)

- **Explore (EDA):** charted the data and found it is balanced; about 84% of attacks use TCP; *src_bytes* is heavily skewed; several error-rate columns are near-duplicates of each other.
- **Clean:** no missing values and no duplicate rows; dropped two columns that never change (*num_outbound_cmds*, *is_host_login*) because they carry no information.
- **Prepare features:** text columns (protocol / service / flag) were one-hot encoded into numbers, and numeric columns were scaled so large values (like bytes) do not dominate small ones.
- **Handle balance:** compared SMOTE and ADASYN against doing nothing — they gave only a marginal lift because the data was already balanced.
- **Train & tune:** four models were trained, each automatically tuned (many settings tried, best kept) and cross-validated 5 ways to confirm the results are stable, not luck.
- **Explain, predict, report:** generated SHAP explanations, scored the unlabelled test file, and assembled this PDF.

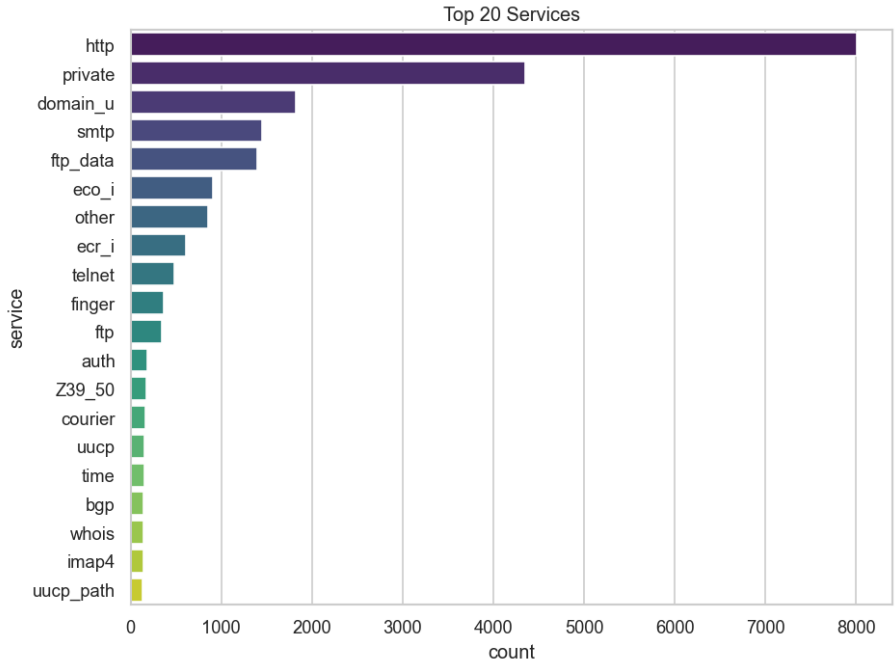
4. Exploratory Data Analysis



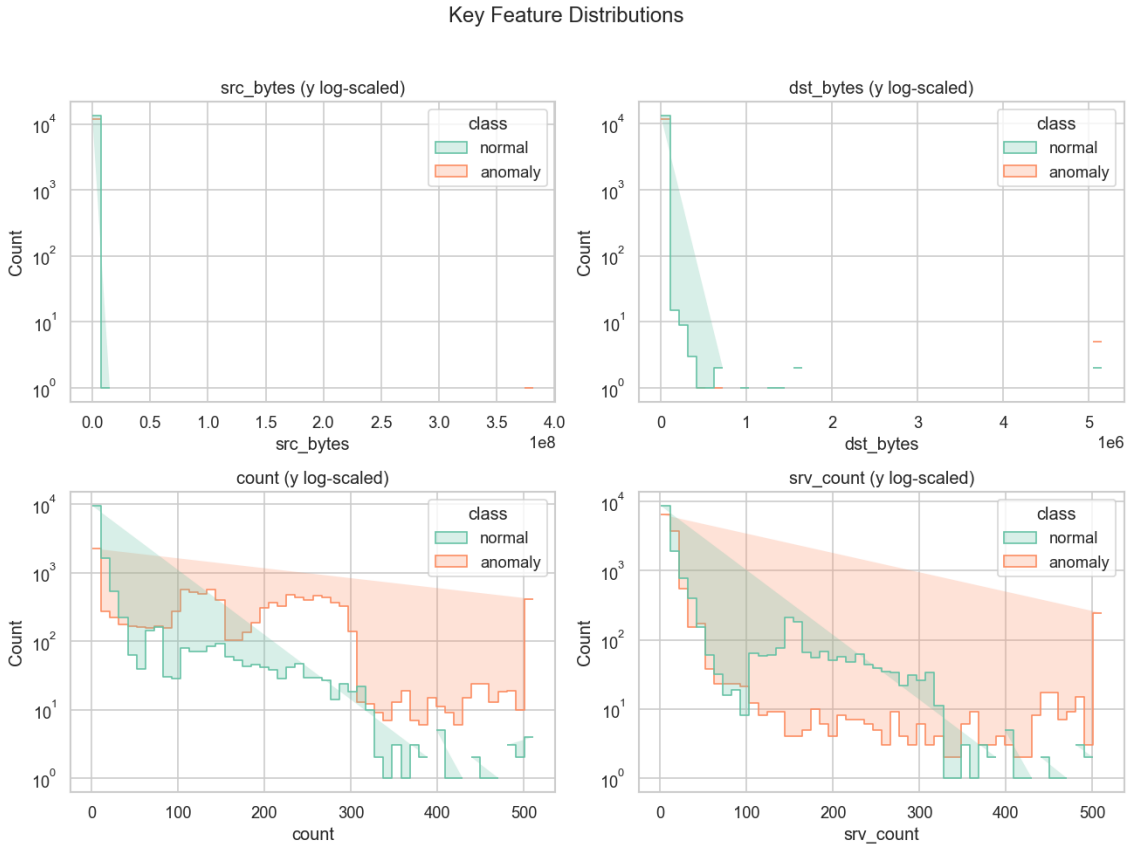
Normal vs anomaly counts — the dataset is well balanced.



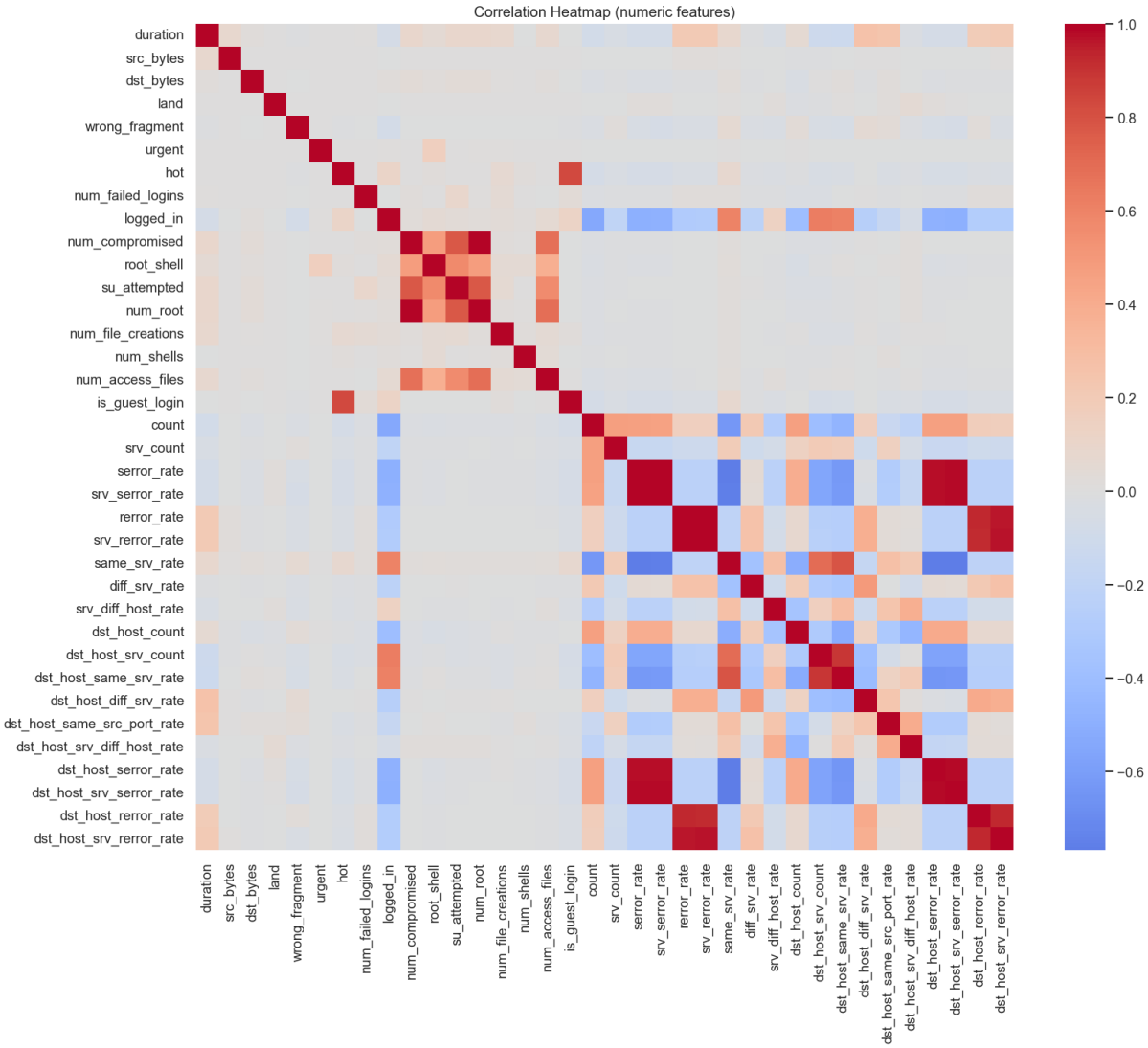
Protocol by class — most attacks ride on TCP.



The 20 most common services.



Key feature distributions (y log-scaled) — note heavy skew and outliers.



Correlation heatmap — bright blocks are redundant, highly-correlated error-rate features.

5. Results — Model Comparison

All four models were scored on the held-out 20% test split (data they never saw during training). XGBoost performed best.

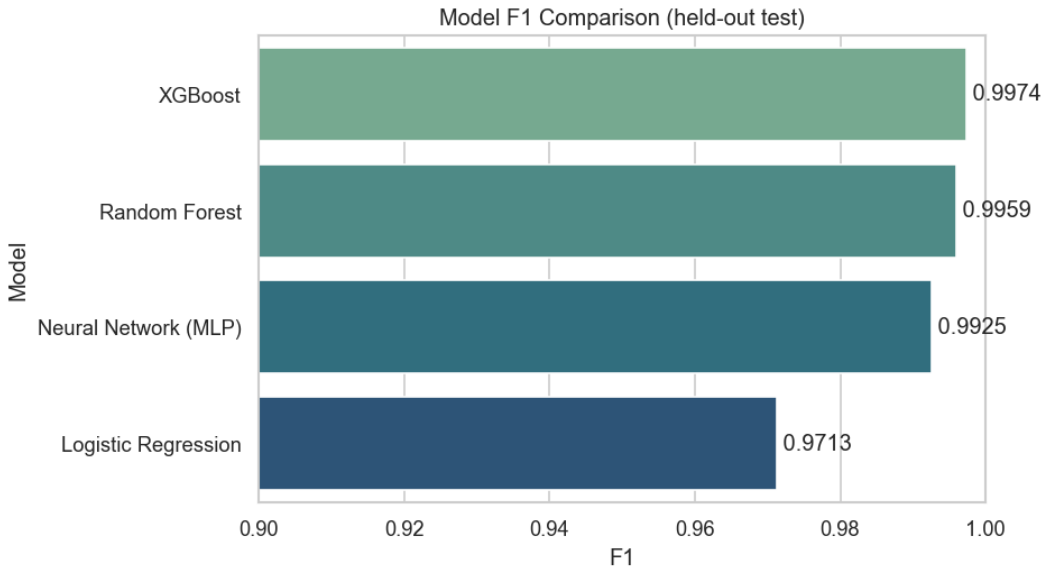
Model	Accuracy	Precision	Recall	F1	ROC AUC	CV F1
XGBoost	**0.9976**	**0.9987**	**0.9962**	**0.9974**	**1.0000**	**0.9978**
Random Forest	0.9962	0.9983	0.9936	0.9959	0.9999	0.9974
Neural Network (MLP)	0.9931	0.9966	0.9885	0.9925	0.9988	0.9931
Logistic Regression	0.9734	0.9772	0.9655	0.9713	0.9944	0.9713

Class-imbalance handling (marginal effect here):

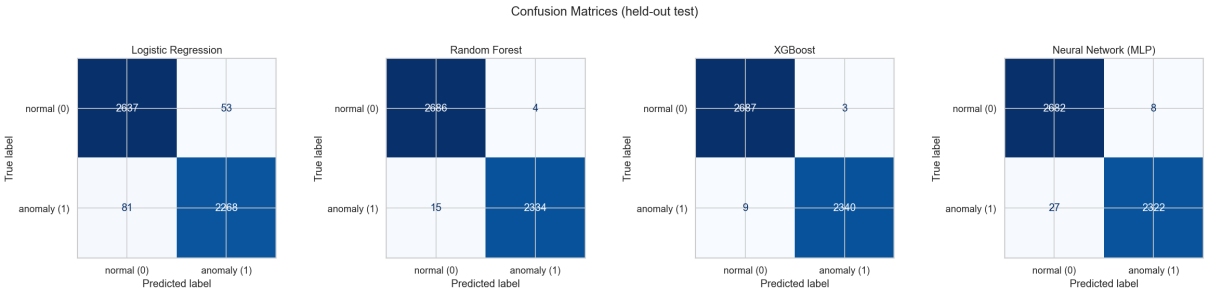
Resampling	XGBoost 5-fold F1
none	0.9977 +/- 0.0004
SMOTE	0.9978 +/- 0.0004
ADASYN	0.9981 +/- 0.0005

What each score means (in IDS terms):

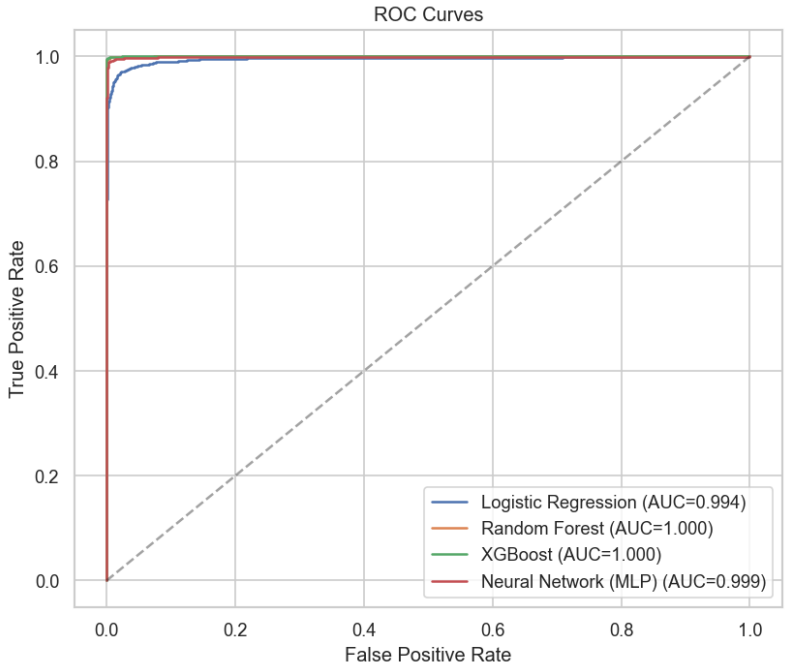
- **Accuracy** — share of all connections classified correctly.
- **Precision** — when the model shouts "attack", how often it is right. High precision = few false alarms, so analysts are not flooded.
- **Recall** — of all real attacks, how many were caught. High recall = few attacks slip through undetected.
- **F1** — a single balanced score combining precision and recall.
- **ROC AUC** — overall ability to tell attack from normal; 1.0 is perfect.
- **CV F1** — average F1 across 5 cross-validation folds, confirming the result is stable rather than a one-off.



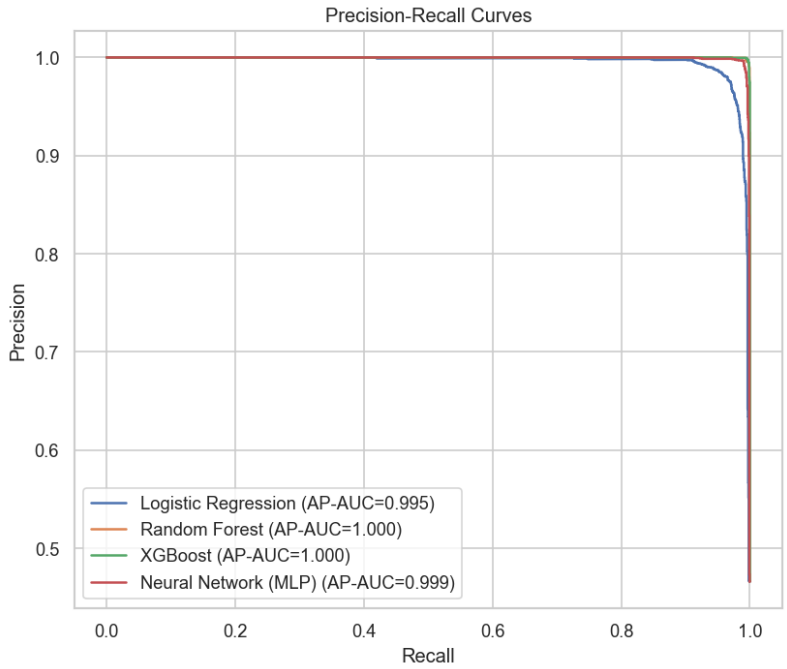
6. Evaluation Plots



Confusion matrices — correct predictions on the diagonal, mistakes off it.



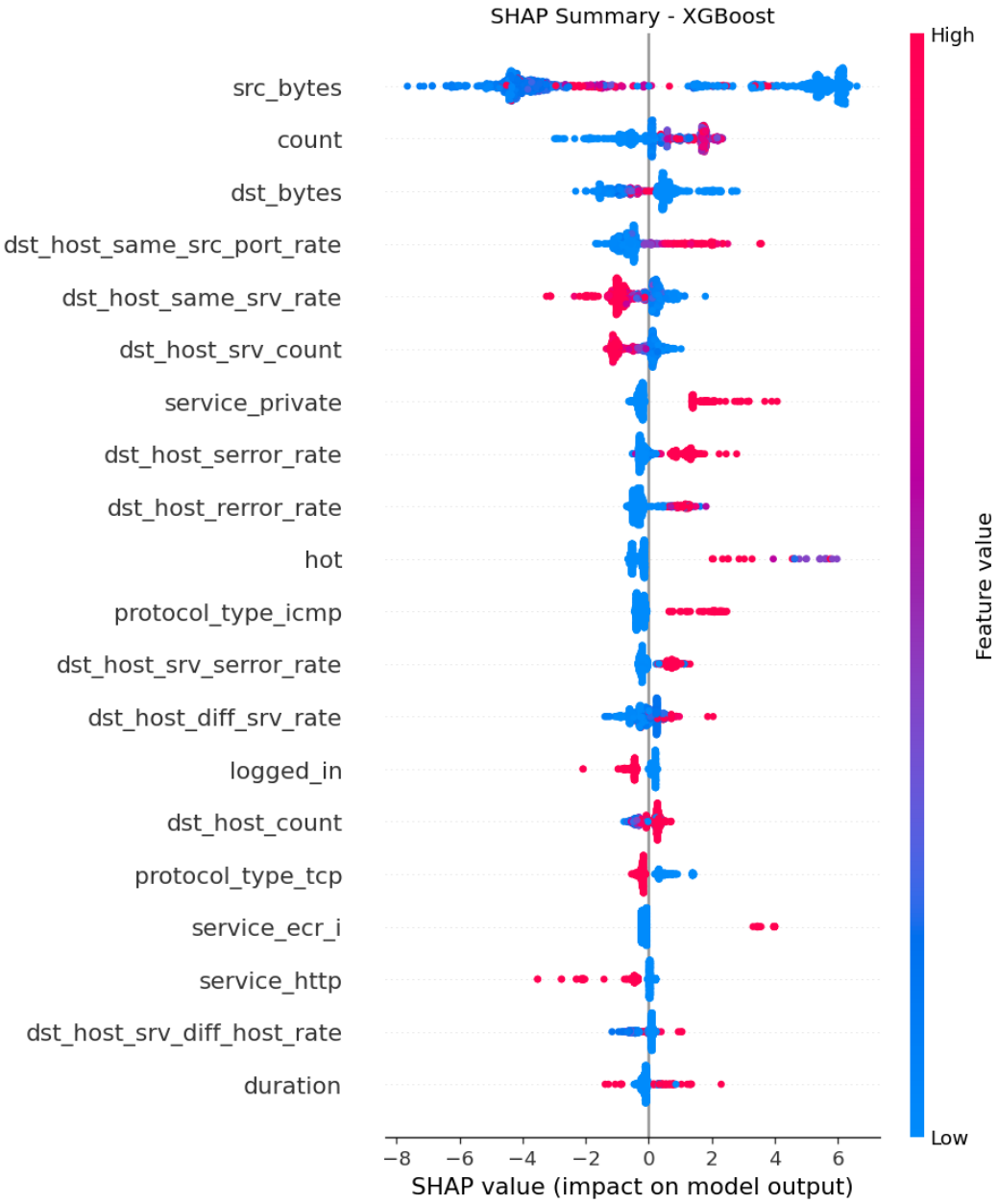
ROC curves — closer to the top-left corner is better.



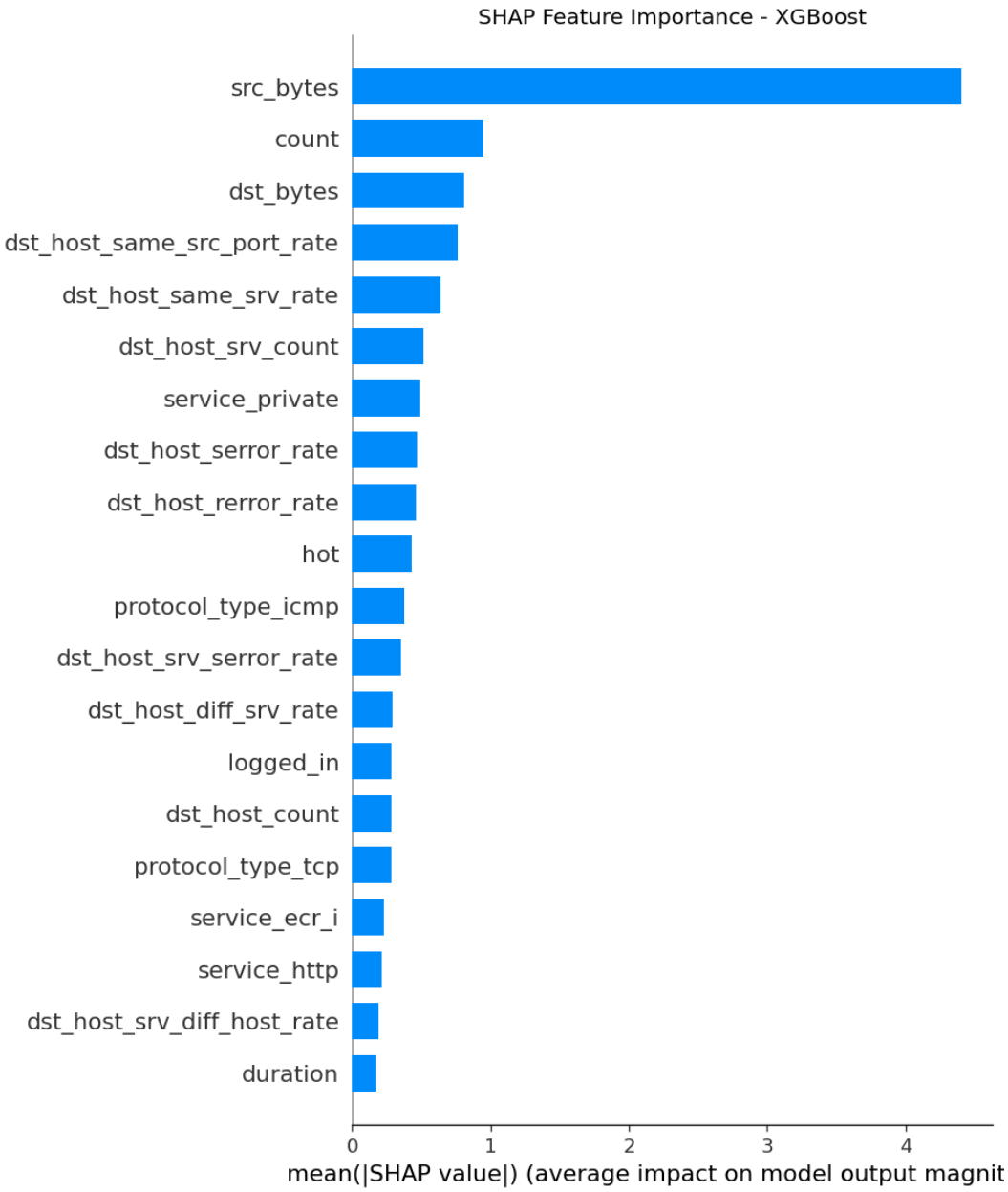
Precision-Recall curves — high and flat is better.

7. Why the Model Decides (SHAP)

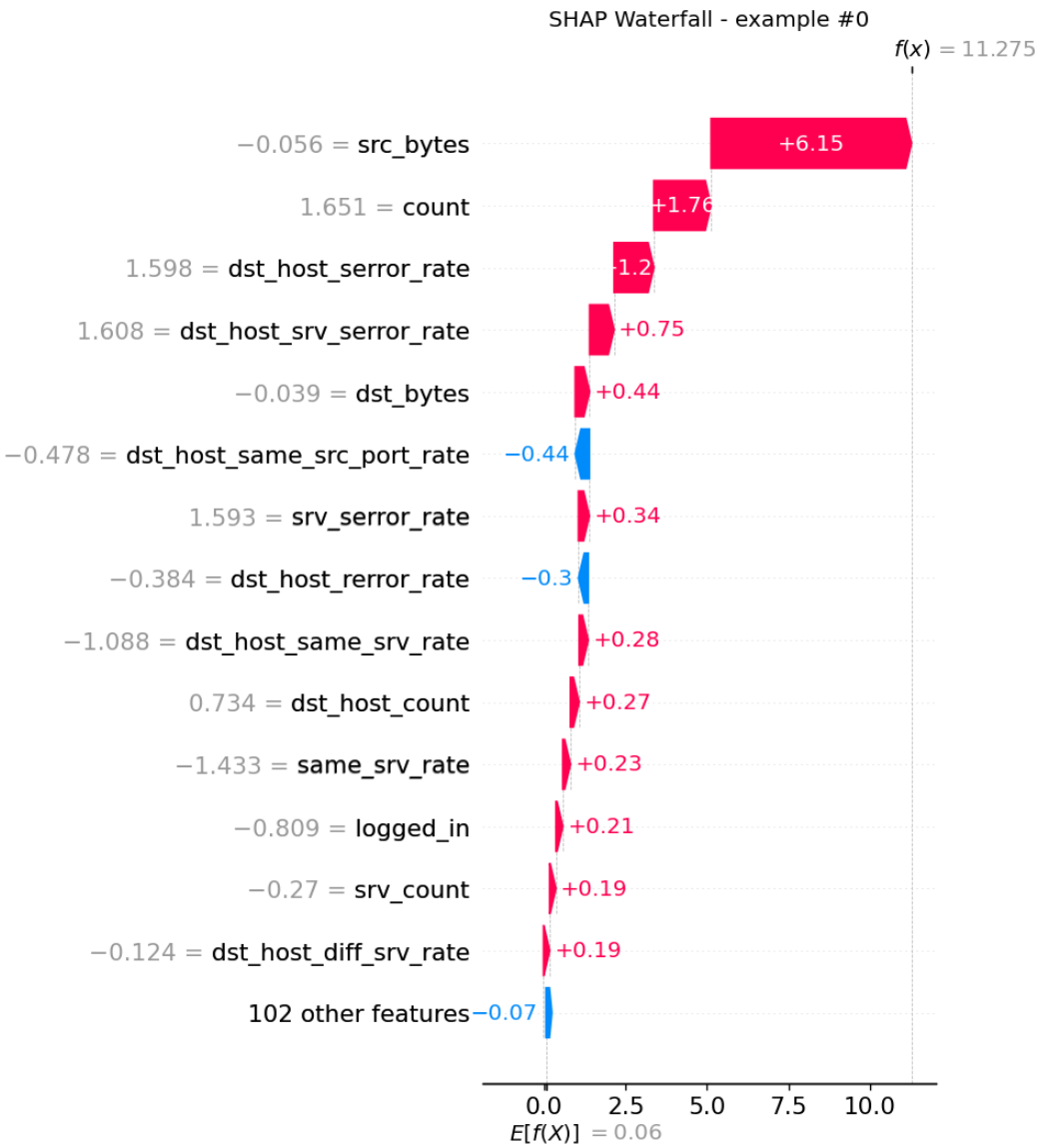
A model that flags attacks without justification is hard to trust. SHAP reveals which features drive each decision. The strongest drivers were **src_bytes** (data sent), **count** (number of connections), and **dst_bytes** (data received), followed by connection error rates, service and protocol — patterns that match real intrusion behaviour, a good sign the model learned genuine signal rather than noise.



SHAP summary — each dot is one connection; colour is the feature value.



Average impact of each feature.



Waterfall — how features pushed one example toward 'attack'.

8. What the Outputs Are

File / folder	What it is
reports/IDS_Report.pdf	This report — charts, tables and explanations.
reports/model_comparison.csv	The results table as a spreadsheet.
reports/figures/	All 13 charts (EDA, ROC, confusion, SHAP).
models/xgb_ids_model.pkl	The winning trained model.
models/best_model.pkl	Copy of the winner, used by the dashboard.
models/random_forest_ids.pkl ...	The other three trained models.
models/preprocessor.pkl	Encoder + scaler that prepares new data identically.
models/metrics.json	All scores in machine-readable form.
outputs/test_predictions.csv	Every unlabelled connection, now scored.
app/streamlit_app.py	Interactive dashboard (live verdict + risk score).

The predictions file adds three columns to each connection: **prediction** (normal / anomaly), **attack_probability** (0–1 confidence) and **risk_score** (0–100). On Test_data.csv the model flagged about 8,476 of 22,544 connections (37.6%) as attacks.

9. An Honest Caveat

Scores are high partly because the test is 'easy'

The ~99.7% scores are real, but the train and test rows come from the same data pool, so the test is relatively easy. Real-world traffic drifts over time, so live performance would be lower. The official harder NSL-KDD test set (KDDTest+) is designed to expose this and is the recommended next step for a realistic estimate.

10. Business Impact

- Real-time, automated detection replaces infeasible manual monitoring.
- Faster detection shrinks attacker dwell time and response time.
- Fewer false alarms (high precision) reduces SOC analyst fatigue.
- Catching more attacks (high recall) lowers breach-related losses.
- SHAP explanations make every alert auditable, supporting analyst trust and incident triage.